Bulk Professional Email Verifier

Prepared for the mycpe ONE marketing team · Jitendra Singh

ACCESS — NO EXPIRY, HOSTED INDEPENDENTLY

URL	Username	Password
https://email-verifier-xs64.onrender.com	**marketing**	**5xdcvu1T1nx**

Runs on Render, not on anyone's laptop. Free-tier trade-off: the instance sleeps after ~15 minutes idle, so the first request afterward can take up to a minute; the database is not on persistent storage, so it resets on that wake-up and on every redeploy. Loaded with synthetic sample data only.

WHAT IT IS

Marketing needs to mail ~250,000 CPA, EA and HR contacts. Sending to a stale list burns sender reputation for every future campaign. This tool checks each address before send and returns a clean, segmented, actionable list — with the reasoning visible, not a black-box verdict.

THE ONE DECISION THAT SHAPES EVERYTHING

It does NOT claim to verify mailbox-level deliverability, and it says so in the interface. True SMTP verification at this volume does not work: Gmail, Outlook and Yahoo do not answer RCPT truthfully; catch-all domains accept every address; and probing 250K mailboxes from one IP is itself what gets a sender blocklisted.

So every address gets one of four statuses plus a 0–100 confidence score and the specific signals behind it, rather than a binary flag nobody can trust:

Valid	Correct syntax, domain accepts mail, nothing flagged. Safe to send.
Risky	Well-formed on a working domain, but the mailbox cannot be confirmed — catch-all domains, shared role inboxes, DNS timeouts. A real category, not an error.
Invalid	Failed syntax, or the domain cannot receive mail at all. Suppress.
Disposable	Throwaway mailbox provider. Deliverable today, gone next week.

Risky is the point. Fold it into Valid and you mail addresses that bounce; fold it into Invalid and you delete thousands of reachable professionals.

HOW EACH ADDRESS IS CHECKED

1. Syntax (RFC 5322, plus typo suggestions like gmial.com -> gmail.com) 2. Domain MX lookup, cached per domain 3. Disposable-provider list (~350 domains, checked in-process) 4. Role-account detection — info@, hr@, billing@ are flagged, never rejected 5. Catch-all detection 6. Optional throttled SMTP probe.

Layers 5 and 6 are off by default. When off, every row carries the reason "Mailbox-level SMTP check not performed", so nothing in the export overstates what was actually verified.

ACCEPTED FILES

CSV, TSV and TXT stream without ever being held in memory. Excel (.xlsx) reads the first worksheet — formula cells resolve to their computed value. Word (.docx) and PDF have no columns, so their text is scanned for addresses and you pick one qualification for the whole document; a repeated address there is one contact mentioned twice, not a duplicate row. Excel/Word/PDF are capped at 25 MB (parsed in memory); CSV at 200 MB.

WHAT YOU CAN DO IN THE APP

- **Upload and map —** drag in a file; columns are auto-matched and you correct them if the guess is wrong.
- **Watch it run —** live row count during reading, then processed/total, percentage, rows-per-second and ETA. The job runs server-side; you can close the tab.
- **Pause and resume —** stop mid-run and continue later. Kill the server outright and the job resumes from the last committed chunk with nothing lost or double-counted.
- **Read the dashboard —** totals and a CPA / EA / HR / Other breakdown, a "why rows were flagged" panel, and a searchable, filterable table of every address with its score and signals.
- **Export —** "Full results" with every status and reason, or "Clean send-list" which is Valid-only and drops duplicate occurrences so nobody is mailed twice.
- **Sample files to try —** CSV samples of various sizes ship in the repo's /sample folder. Excel, Word and PDF uploads are supported and were tested extensively before this handoff; this instance just has nothing pre-loaded, since a redeploy would wipe it anyway.

WHAT IT DELIBERATELY DOES NOT DO

- **Confirm a mailbox exists.** See the reasoning on page 1. A production next step is a paid API (ZeroBounce, Kickbox) called only for rows marked Risky.
- **OCR scanned PDFs.** An image-only PDF is rejected with that specific reason rather than silently returning zero addresses.
- **Read legacy .xls.** The only npm library that reads the pre-2007 binary format carries an unpatched high-severity advisory; not worth pointing at uploaded files. .xlsx is supported.
- **Send email.** It verifies and exports. Sending stays in your existing platform.
- **Send your data anywhere.** No analytics, no error tracker, no CDN. The disposable-domain list is bundled rather than fetched. The only outbound traffic is the DNS that verification inherently requires.

HOW I APPROACHED THIS AS PM

- **Started from the failure mode, not a feature list —** a stale list burns sender reputation, so every design call (four statuses, visible reasoning, resumability) traces back to preventing that.
- **Sequenced by risk —** verification logic first, then the storage gap I flagged in my own README, then wider file-format coverage, then a reviewable public demo. Each phase shipped complete before the next began.
- **Tested adversarially before calling anything done —** real browser uploads through a real public tunnel, not unit tests alone. That is what caught two production bugs a local test run would have missed.
- **Treated PII handling as a requirement from commit one,** not a pass at the end — encryption, redacted logs and export-injection protection were built alongside the verification logic itself.

SECURITY AND DATA HANDLING

The list is real contact data for professionals in a compliance-driven industry, so: the upload is encrypted (AES-256-GCM) the moment it arrives and the plaintext temp file is shredded before you even reach the mapping screen. Email addresses are ciphertext in the database — grepping the raw file recovers no addresses and no local parts. Logs contain job IDs and counts only; zero addresses were found in the logs across a full upload -> verify -> export cycle. Everything sits behind a login. Exports are protected against CSV formula injection. Jobs auto-delete on a retention window (7 days) — moot on this particular instance, since the database is not on persistent storage and does not survive a redeploy or a spin-down/restart anyway.

Domain, segment and status are stored readable on purpose — that is what lets the database index and aggregate. A row of {domain, qualification, status} identifies nobody; the same row with the local part would.

BUILT WITH, AND HOW IT WAS VERIFIED

Node.js/Express, React/Vite, SQLite (WAL) with versioned migrations. Verification runs off the request cycle in chunks, with per-domain DNS rate limiting so a fast pipeline cannot hammer one corporate mail server.

416,144 rows	end to end in 176s (44s read, 131s verify) — about 105s for a 250K list
241 ms	worst API latency during that run, with zero failed requests
61 automated tests	covering syntax, injection, encryption, resume-without-double-count, the SMTP layers against a mock server, and every file format
0 vulnerabilities	in production dependencies (npm audit)
Crash-tested	hard-killed at chunk 52 of 157; restarted, resumed, and finished with counts identical to an uninterrupted run

---

Two things worth knowing about this instance

It is hosted on Render's free tier, chosen deliberately to avoid putting a card on file for a review link. Two trade-offs come with that: the instance spins down after ~15 minutes idle (the next request wakes it, slowly), and there is no persistent disk, so uploaded jobs do not survive a redeploy or a spin-down/restart. Neither is a code limitation — the app already supports resumable, persistent jobs, as the crash test above shows; the render.yaml just needs a disk and a paid plan added back to get that here too.

This instance holds synthetic sample data only — no real contact list has been loaded onto a public URL.